

Projet de Recherche (PRe)

Spécialité : Mathématiques

Année scolaire : 2024–2025

Algorithmes de placement optimisés de drones pour la conception de réseaux de communication

Non Confidentialité

Auteur : Haobin JIANG

Promotion : ENSTA 2025

Tuteur ENSTA : Zacharie ALES

Tuteur organisme d'accueil : Zacharie
ALES

Stage effectué du 26/05/2025 au 01/08/2025

Nom de l'organisme d'accueil : ENSTA Paris

Adresse : 828, boulevard des Maréchaux – 91120 Palaiseau

Résumé :

Dans le problème de déploiement de drones à haute altitude (HAPS), plusieurs approches ont été proposées pour maximiser le nombre de clients couverts tout en respectant les contraintes de communication et de couverture. Parmi celles-ci, on distingue notamment les modèles linéaires en nombres entiers (PLNE), les modèles quadratiques non linéaires (PQNE), ainsi que les algorithmes gloutons.

Ces méthodes présentent des avantages complémentaires : le PLNE offre une structure qui permet une résolution exacte rapide, le PQNE permet de placer les drones n'importe où dans l'espace, et l'algorithme glouton assure une rapidité d'exécution précieuse en phase d'initialisation.

L'objectif principal de ce travail est de proposer une stratégie hybride combinant ces trois approches, dans le but d'améliorer la qualité de la solution obtenue dans un temps limité. En tirant parti de la structure du PLNE, de la richesse descriptive du PQNE, et de l'efficacité du glouton, nous construisons un cadre d'optimisation mixte, à la fois robuste et efficace. Les résultats expérimentaux montrent que cette approche intégrée permet d'obtenir des performances supérieures à celles des méthodes prises isolément, en particulier dans les instances de grande taille.

Mots-clés : déploiement de drones, programmation linéaire en nombres entiers (PLNE), optimisation quadratique (PQNE), algorithme glouton, connectivité, couverture

Abstract :

In the high-altitude platform station (HAPS) deployment problem, several approaches have been proposed to maximize the number of covered clients while respecting communication and coverage constraints. Among them, we distinguish integer linear programming (ILP), nonlinear quadratic models (PQNE), and greedy algorithms.

These methods offer complementary advantages : ILP provides a structured and fast exact resolution ; PQNE allows placing drones freely in space ; and the greedy algorithm ensures fast initialization.

The main objective of this work is to propose a hybrid strategy that combines these three approaches to improve solution quality under limited time constraints. By leveraging the structure of ILP, the descriptive power of PQNE, and the efficiency of the greedy approach, we construct a mixed optimization framework that is both robust and efficient. Experimental results show that this integrated approach yields better performance than each individual method, especially on large-scale instances.

Keywords : drone deployment, integer linear programming (ILP), quadratic optimization (PQNE), greedy algorithm, connectivity, coverage

Remerciements

Je tiens tout d'abord à exprimer ma profonde gratitude à Monsieur Zacharie ALES pour son encadrement patient et rigoureux tout au long de ce stage. Il a toujours pris le temps de répondre à chacune de mes questions de manière claire et constructive, ce qui a grandement facilité ma progression.

Je remercie également l'équipe du laboratoire UMA (ENSTA Paris) pour son accueil chaleureux. Que ce soit pour les démarches administratives, la mise à disposition des équipements ou l'accès aux serveurs de calcul, tout a été organisé de manière efficace et bienveillante. L'environnement de travail y est à la fois stimulant et convivial.

Enfin, je remercie sincèrement tous les enseignants, doctorants et camarades qui m'ont apporté leur aide ou leurs conseils à différents moments de ce travail.

Contents

Résumé et Abstract	1
Remerciements	2
1 Présentation du laboratoire	4
2 Contexte	4
2.1 Présentation du problème	4
2.2 Programmation Linéaire en Nombres Entiers(PLNE)	5
2.2.1 Algorithmes gloutons	6
2.2.2 Callback (Inégalités violés) et Cutting Plane	7
2.3 Programmation Quadratique en Nombres Entiers (PQNE)	8
3 Travaux réalisés	11
3.1 Méthode combinant PLNE et PQNE	11
3.1.1 Valeur propre	11
3.2 Implémentation	12
3.3 Résultats numériques	13
A L'exemple de l'instance	19
B Les code essentiels	22

1 Présentation du laboratoire

J'ai réalisé mon stage au sein de L'Unité de Mathématiques Appliquées (UMA). L'unité mène des missions d'enseignement et de recherche à l'ENSTA en association avec le CNRS, Inria et EDF R&D. Elle est impliquée dans le cycle ingénieur ENSTA et dans les masters d'IP-Paris. Les thèmes de recherche comprennent la modélisation et la simulation numérique, le contrôle et l'optimisation, le calcul scientifique, la recherche opérationnelle et la science des données, les équations aux dérivées partielles et les probabilités.

En même temps, le laboratoire UMA offre également une puissance de calcul importante et un soutien logiciel solide, et organise régulièrement des séminaires académiques.

UMA regroupe trois équipes de recherche : OC (Optimisation et Commande), POEMS (Propagation d'Ondes : Étude Mathématique et Simulation), et IDEFIX (Incertitude, Données, Efficacité, Fiabilité et approXimation). J'ai été intégré à l'équipe OC, spécialisée en optimisation continue, discrète et combinatoire, notamment dans le cadre des problématiques liées à la couverture, la connectivité et la planification.

2 Contexte

2.1 Présentation du problème

L'objectif principal de ce stage était de développer une nouvelle approche pour résoudre un problème de placement de drones. Il s'agissait notamment d'explorer des formulations hybrides combinant des méthodes exactes (de type PLNE) avec des techniques continues ou quadratiques, tout en tenant compte des contraintes structurelles et de performance propres aux applications visées.

Contexte et modélisation

On considère un problème d'optimisation combinatoire lié au déploiement de drones pour établir un réseau de communication sur un théâtre d'opération. Trois types d'éléments sont présents :

- des **unités terrestres**, dont les positions sont connues ;
- une **base avancée**, positionnée en un point fixe ;
- des **drones**, à déployer à une altitude fixe sur un ensemble de points à déterminer.

Les éléments peuvent communiquer entre eux à travers différents **types de communication**, chacun associé à un rayon spécifique. Les drones peuvent emporter des **relais de communication**, limités par leur charge utile et leur puissance électrique maximale. Chaque relais est caractérisé par un poids, un rayon et une consommation électrique.

Remarque importante : En pratique, chaque unité (client ou site potentiel) possède une certaine altitude propre. Toutefois, afin de simplifier l'étude et réduire la complexité des calculs géométriques, nous faisons l'hypothèse que toutes les unités se trouvent à la même altitude. Cela pourrait influencer sur la portée effective des relais.

Objectif d'optimisation

L'objectif est de déterminer :

- les **positions de p drones** ;
- les **relais à embarquer** pour chaque drone ;

de manière à :

- établir un **réseau connexe** reliant la base, les drones et un sous-ensemble d'unités ;
- **maximiser le nombre d'unités couvertes**.

Deux sous-problèmes apparaissent :

1. Le choix des relais (proche d'un problème de bin packing) ;
2. Le choix des positions des drones (problème de couverture-connectivité).

Dans ce travail, on s'intéresse uniquement au deuxième sous-problème et on suppose que tous les drones et les unités utilisent le même type de relais.

Cas particulier étudié

Dans la version restreinte du problème :

- un seul type de communication est considéré dans ce stage ;

— chaque drone possède un relais fixe, donc seul son positionnement est à déterminer.

Formellement, le problème est :

Étant donné les coordonnées des unités, ainsi que les rayons de couverture R_{cov} et de communication R_{com} , trouver les p positions de drones qui forment un réseau connexe avec la base et maximisent le nombre d'unités couvertes.

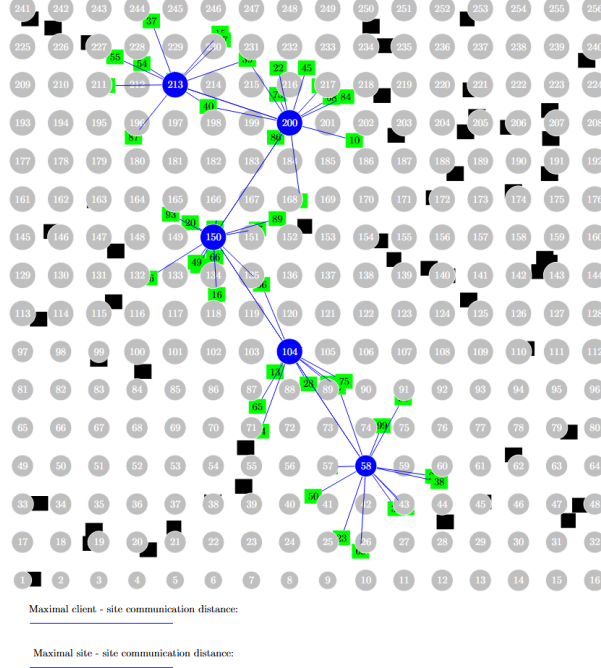


FIGURE 1 – Un exemple d'instance résolue pour $p = 5$

Légende(Figure 1 et 4) :

- Cercles bleus : drones déployés.
- Cercles gris : positions non utilisées.
- Carrés noirs : clients non couverts.
- Carrés verts : clients couverts.
- Lignes vertes : relations de couverture drone-client.
- Lignes bleues : connexions de communication entre drones.

2.2 Programmation Linéaire en Nombres Entiers(PLNE)

Méthode de résolution et modélisation mathématique

Ce problème a été modélisé [2] par un PLNE dans lequel les positions possibles des drones sont représentées par un ensemble discret sous la forme d'une grille. Ce problème peut être résolu via des solveurs spécialisés tels que CPLEX en utilisant la méthode du *Branch and Bound*. Cela est possible uniquement si la taille de la formulation (nombre de variables et de contraintes) reste raisonnable.

Notations et hypothèses

Soit $\mathcal{U}$ l'ensemble des unités et $\mathcal{P}$ l'ensemble des positions possibles pour les drones. On connaît les coordonnées de chaque unité et de chaque position. Après de ça, on peut ainsi calculer la distance entre n'importe quelle paire de points. Le nombre maximal de drones est noté p .

Chaque drone est identique et possède :

- un rayon de couverture R_{cov} , dans lequel il peut couvrir des unités ;
- un rayon de communication R_{com} , dans lequel il peut communiquer avec d'autres drones.

La connectivité entre drones est modélisée par un **graphe non orienté**, dont les sommets sont les positions de drones et les arêtes relient deux sommets si la distance entre eux est inférieure à R_{com} .

Pour modéliser la connexité, on impose que la position des drones forme un arbre couvrant dans lequel chaque arête correspond à une distance inférieure à R_{com} . Dans ce stage, nous imposons cette contrainte par une **modélisation MTZ**.

Formulation MTZ

Nous introduisons les variables suivantes :

- y_j : variable binaire valant 1 si et seulement si un drone est placé à la position $j \in \mathcal{P}$;
- $x_{ij}, i \neq j$: variable binaire valant 1 si l'arc (i, j) est sélectionné pour faire partie de l'arborescence couvrante. Cela sous-entend que des drones sont placés en i et j , et que la distance entre ces positions est inférieure au rayon de communication ;
- c_u : variable binaire valant 1 si et seulement si l'unité $u \in \mathcal{U}$ est couverte ;
- t_j : entier associé à $j \in \mathcal{P}$ pour assurer l'existence d'un ordre topologique.

La formulation pour le problème *(Loc)* s'écrit comme suit :

$$\begin{aligned}
 \text{(MTZ)} \quad \left\{ \begin{array}{ll} \max & \sum_{u \in \mathcal{U}} c_u \\ \text{s.c.} & c_u \leq \sum_{j \in \mathcal{P}, d_{uj} \leq R_{\text{cov}}} y_j \quad \forall u \in \mathcal{U} \quad (1) \\ & \sum_{j \in \mathcal{P}} y_j \leq p \quad (2) \\ & x_{ij} = 0 \quad \forall i, j : d_{ij} > R_{\text{com}} \quad (3) \\ & x_{ji} \leq y_j \quad \forall j, i \in \mathcal{P} \quad (4) \\ & \sum_{i \in \mathcal{P}} x_{ij} \leq y_j \quad \forall j \in \mathcal{P} \quad (5) \\ & \sum_{i \in \mathcal{P}} \sum_{j \in \mathcal{P}} x_{ij} = \sum_{j \in \mathcal{P}} y_j - 1 \quad (6) \\ & t_j \geq t_i + 1 - p(1 - x_{ij}) \quad \forall j, i \in \mathcal{P} \quad (7) \\ & y_j \in \{0, 1\}, c_u \in \{0, 1\}, x_{ij} \in \{0, 1\}, t_j \geq 0 \end{array} \right.
 \end{aligned}$$

- (1) impose que l'unité u soit marquée comme couverte uniquement si un drone est placé à une position capable de la couvrir ;
- (2) limite le nombre total de drones déployés à p ;
- (3) interdit les arcs entre des positions trop éloignées ;
- (4) et (5) assurent qu'un arc ne peut être activé que si les deux positions extrémités sont occupées par des drones ;
- (6) garantit que le sous-graphe est une arborescence avec $p - 1$ arêtes (connexité entre les drones) ;
- (7) assure l'ordre topologique des sommets, nécessaire pour éviter les cycles dans l'arborescence.

La contrainte (7) permet de forcer $x_{ij} = 1$ seulement si $t_j = t_i + 1$, ce qui construit une arborescence dirigée. Cette formulation est connue sous le nom de modélisation MTZ. Cette contrainte est souvent non contraignante dans les cas où tous les drones ne sont pas utilisés.

Par ailleurs, afin de ne pas restreindre les drones à des positions fixes sur une grille, une formulation quadratique continue a également été introduite. Cette approche permet d'explorer un espace de recherche plus flexible, en contournant les limitations inhérentes à la discrétisation spatiale.

2.2.1 Algorithmes gloutons

À mesure que la taille du problème augmente, la résolution exacte des modèles de PLNE décrits plus haut peut devenir trop coûteuse en temps de calcul. Pour obtenir des solutions dans un délai limité sur les grandes instances, deux heuristiques gloutonnes ont été introduites [1] : d'abord une version déterministe, puis une variante randomisée. Comme on le verra en Section 3.3, ces méthodes produisent des solutions de bonne qualité avec des temps de calcul tout à fait raisonnables.

Notations. Soit P l'ensemble des positions possibles et p le nombre de drones à ouvrir. On impose la connexité du réseau via une portée L : pour un ensemble $S \subseteq P$, on définit $P_S := \{j \in P : \text{dist}(j, S) \leq L\}$ (candidats connectés à S). La qualité d'une solution S est évaluée par une fonction d'objectif $F(S)$ (par ex. le nombre d'unités couvertes).

Fonctions d'évaluation (au choix). L'algorithme va itérativement ouvrir des sites jusqu'à ce qu'il y en ait p . Pour choisir le site à ouvrir, l'algorithme va se baser sur une fonction d'évaluation $f : P \mapsto \mathbb{R}$. Plus $f(j)$ est élevé, plus le site j aura de chances d'être ouvert. Voici les deux fonctions considérées :

$$f_{\text{PL}}(j) = \bar{y}_j, \quad \text{où } (\bar{x}, \bar{y}) \text{ est la solution de la relaxation linéaire (MTZ) du PLNE,} \quad (1)$$

$$f_L(j) = \#\{\text{unités couvertes par } S \cup \{j\}\} - \#\{\text{unités couvertes par } S\}. \quad (2)$$

Algorithm 1: Glouton déterministe paramétré par f (utiliser (1) ou (2))

Entrées: Instance I ; ensemble P ; entier p ; portée L ; fonction d'évaluation $f : P \rightarrow \mathbb{R}_+$.

Sortie: Ensemble $S \subseteq P$ de taille p formant un réseau connexe.

```

1  $S \leftarrow \emptyset$ ;
2  $P_S \leftarrow P$ ; // au départ, aucun site ouvert
3 while  $|S| < p$  et  $P_S \neq \emptyset$  do
4    $s^* \leftarrow \arg \max_{s \in P_S} f(s)$ ;
5    $S \leftarrow S \cup \{s^*\}$ ;
6    $P_S \leftarrow \{j \in P : \text{dist}(j, S) \leq L\}$ ; // mise à jour de la bande connectée
7 return  $S$ ;
```

L'algorithme glouton déterministe cherche, à chaque itération, la position de drone qui maximise la fonction d'évaluation tout en respectant les contraintes de faisabilité (notamment la couverture des clients et la connexité entre les drones). Pour une instance donnée, son comportement est donc entièrement déterminé par la fonction d'évaluation et l'ordre de sélection : la solution produite est toujours la même d'un lancement à l'autre, ce qui illustre le caractère déterministe de la méthode.

Algorithm 2: Glouton randomisé (répétitions avec tirage pondéré par f)

Entrées: Instance I ; ensemble P ; entier p ; portée L ; fonction $f : P \rightarrow \mathbb{R}_+$; nombre d'itérations $l_{\max}$.

Sortie: Meilleure solution S_{best} trouvée.

```

1  $S_{\text{best}} \leftarrow \text{GLOUTONDÉTERMINISTE}(I, P, p, L, f)$ ; // Algorithme 1
2 for  $\ell \leftarrow 1$  to  $l_{\max}$  do
3    $S \leftarrow \emptyset$ ;
4    $P_S \leftarrow P$ ;
5   while  $|S| < p$  et  $P_S \neq \emptyset$  do
6     Tirer  $s$  dans  $P_S$  avec probabilité proportionnelle à  $f(s)$ ;
7      $S \leftarrow S \cup \{s\}$ ;
8      $P_S \leftarrow \{j \in P : \text{dist}(j, S) \leq L\}$ ;
9   if  $F(S) > F(S_{\text{best}})$  then
10     $S_{\text{best}} \leftarrow S$ 
11 return  $S_{\text{best}}$ ;
```

L'algorithme glouton randomisé (Algorithme 2) repose sur une version modifiée de l'algorithme déterministe, dans laquelle la sélection du site à chaque étape est remplacée par un tirage aléatoire pondéré selon la fonction d'évaluation f , au lieu de toujours choisir le site qui maximise directement f .

Cette modification introduit de la diversité dans les solutions générées, en évitant de systématiquement sélectionner le site le plus prometteur. Si la solution courante S dépasse la meilleure solution S_{best} trouvée jusque-là, cette dernière est mise à jour. Le processus est répété jusqu'à ce que le nombre maximal d'itérations soit atteint.

2.2.2 Callback (Inégalités violés) et Cutting Plane

Dans la résolution d'un PLNE par la méthode Branch & Bound, le nombre de branches devenant trop élevé et l'élagage trop complexe, nous avons retenu deux schémas d'accélération :

1. **Plans coupants (cutting planes)** : dès le démarrage de l'élagage, au nœud racine, on enrichit la relaxation en y ajoutant des inégalités supplémentaires tirées issues de familles d'inégalités valides.
2. **Callbacks** : le principe est similaire, mais l'ajout d'inégalités s'effectue à chaque nœud (en limitant toutefois le nombre d'inégalités insérées à chaque appel), ce qui restreint progressivement l'espace de la relaxation et rapproche plus vite les bornes inférieure et supérieure.

Nous présentons ici les deux types d'inégalités mises en œuvre au cours de ce stage.

- **Inégalités de clique** : Ces inégalités identifient un ensemble C de clients tel que tout couples de clients c_1, c_2 de C ne peuvent pas être couverts par un même HAPS. Ainsi, dans une solution, au maximum p de ces clients peuvent être couverts. Formellement, si $C \subseteq \mathcal{C}$ est un sous-ensemble de clients tels que la distance entre eux est supérieure à $2r_{\text{couv}}$, alors :

$$\sum_{i \in C} \sum_{k \in \mathcal{S}} c_{ik} \leq p$$

Ces inégalités sont valides car toute solution admissible doit respecter les contraintes de couverture géographique induites par le rayon de couverture et le nombre limité de drones.

- **Inégalités de clusters incompatibles** : Elles permettent de modéliser le fait que certains groupes d'éléments (clients ou sites) ne peuvent pas être couverts simultanément dans une même solution, car ils sont trop éloignés les uns des autres pour être connectés. Mathématiquement, si $S_1, S_2 \subseteq \mathcal{S}$ sont deux groupes de clients mutuellement incompatibles qui signifie que tout couples s_1 de S_1 et s_2 de S_2 sont trop éloignés pour être dans la même solution.

Il faut donc que leur distance soit $> (p-1)R_{\text{com}} + 2r_{\text{couv}}$. En effet, si on fait une ligne droite la plus grande possible avec p drones, la distance entre le 1er et le dernier drone est $(p-1)R_{\text{com}}$. Il faut d'abord ajouter une variable binaire $y_{j,S}$ pour chaque élément j dans un cluster S , qui vaut 1 si le client j est couvert. On également défini une variable binaire z_S qui vaut 1 si le cluster S est utilisé (au moins un client de cet ensemble est couvert), puis lier chaque $y_{j,S}$ à z_S via la contrainte :

$$\sum_{j \in S} y_{j,S} \geq z_S$$

et finalement imposer :

$$\sum_S z_S \leq 1$$

Ces contraintes sont valides car elles assurent que le graphe de communication des drones reste connexe : activer des sites trop éloignés violerait la contrainte de connectivité du réseau.

Lors du processus de callback, le solveur n'autorise pas l'ajout de nouvelles variables (incompatible), alors qu'avec la méthode des plans coupants, ces ajouts sont possibles. Ainsi, nous avons introduit une version plus faible des inégalités d'incompatibilité présentées précédemment : au lieu d'imposer des inégalités précises pour chaque groupe incompatible S_k , nous avons utilisé une version globale plus souple, de la forme :

$$\sum_{u \in \bigcup_{k=1}^K S_k} x_u \leq M$$

où M représente la taille maximale parmi les groupes S_k . Cette contrainte ne garantit pas que chaque groupe est traité individuellement, mais elle limite globalement le nombre d'éléments incompatibles pouvant être activés simultanément.

Cette version relâchée est bien adaptée au cadre du callback, car elle permet d'introduire rapidement des coupes valides dans la relaxation courante sans complexifier excessivement la structure du modèle. Elle agit comme un filtre général, évitant des configurations manifestement non admissibles, tout en restant compatible avec les contraintes d'efficacité computationnelle imposées par le solveur.

Les inégalités de clique et de clusters incompatibles présentent la même limitation : les inégalités ne peuvent être découvertes que lorsque le nombre de drones est faible, mais dans ce cas précis, la résolution directe du problème s'avère plus efficace, évitant ainsi le coût de découverte et de validation des inégalités.

2.3 Programmation Quadratique en Nombres Entiers (PQNE)

Bien que la formulation en PLNE offre un avantage notable en termes de rapidité de résolution, sa performance finale dépend fortement de la discrétisation du plan en grille ainsi que de la répartition spatiale des clients. En particulier, on observe que plus le nombre de positions potentielles est élevé, plus la solution optimale du PLNE peut être de bonne qualité. Toutefois, cette amélioration s'accompagne d'une augmentation significative du temps de calcul, ce qui peut rapidement devenir prohibitif pour les instances de grande taille.

Sur la base de la formulation PLNE, il est possible de conserver la définition des variables inchangée, à l'exception des positions des drones, que l'on rend alors **continues**. Autrement dit, les drones ne sont

plus limités aux points d'une grille, mais peuvent être placés librement sur tout le plan. Cette approche permet d'obtenir des résultats plus précis ainsi qu'une solution globalement optimale.

Ainsi, il n'est plus nécessaire d'introduire la variable y_j pour indiquer si un drone est placé sur un point de la grille. À la place, on utilise les variables α et β pour représenter respectivement les coordonnées horizontales et verticales des drones.

Formulation Quadratique

Variables :

- c_{ik} : variable indiquant si le client i est couvert par le drone k .
- z_i : variable indiquant si le client i est couvert par au moins un drone.
- $x_{kk'}$: variable indiquant s'il existe un lien de communication entre les drones k et k' .
- α_k : coordonnée horizontale (abscisse) du drone k .
- β_k : coordonnée verticale (ordonnée) du drone k .
- t_k : indice topologique du drone k dans l'arborescence couvrante (utilisé pour la formulation MTZ).

$$\left\{ \begin{array}{ll} \max & \sum_{i=1}^n z_i \\ \text{s.c.} & (\alpha_k - \alpha_{k'})^2 + (\beta_k - \beta_{k'})^2 \leq R_{\text{com}}^2 x_{kk'} + M(1 - x_{kk'}) \quad \forall k \neq k' \quad (8) \\ & (\alpha_k - x_i)^2 + (\beta_k - y_i)^2 \leq (R_{\text{cov}}^2 - L^2) c_{ik} + \varepsilon(1 - c_{ik}) \quad \forall i, k \quad (9) \\ & z_i \leq \sum_{k=1}^p c_{ik} \quad \forall i \in \{1, \dots, n\} \quad (10) \\ & \sum_{k \neq k'} x_{kk'} = p - 1 \quad (11) \\ & \sum_{k' \neq k} x_{kk'} \leq 1 \quad \forall k \in \{1, \dots, p\} \quad (12) \\ & t_{k'} \geq t_k + 1 - p(1 - x_{kk'}) \quad \forall k \neq k' \quad (13) \\ & c_{ik}, x_{kk'} \in \{0, 1\}, \quad z_i \in [0, 1], \quad \alpha_k \in [x_{\min}^S, x_{\max}^S], \quad \beta_k \in [y_{\min}^S, y_{\max}^S], \quad t_k \geq 0 \end{array} \right.$$

Les contraintes (8) à (13) ont été introduites pour garantir la faisabilité géométrique et la connectivité du réseau de drones. Leur rôle est détaillé ci-dessous :

- **Contraintes (8) et (9)** : Ces deux contraintes assurent que la distance entre les entités couvertes reste compatible avec les portées physiques définies :
 - la contrainte (9) impose que la distance entre deux drones activés soit inférieure ou égale à R_{cov} (portée de couverture), assurant une couverture effective des unités cibles ;
 - la contrainte (8) impose que tout site ne puisse être activé que s'il est dans la portée de communication R_{com} d'un drone actif.
- **Contrainte (10)** : Cette contrainte précise que pour qu'un site soit couvert, il doit nécessairement se trouver dans la zone de couverture d'au moins un drone activé. Cela évite d'attribuer une couverture à des sites inaccessibles.
- **Contraintes (11) à (13)** : Ces contraintes implémentent la structure MTZ (Miller–Tucker–Zemlin) classique afin d'assurer que les drones actifs forment un graphe connexe. Plus précisément :
 - elles introduisent des variables auxiliaires d'ordre pour éliminer les cycles et garantir que la structure résultante est un arbre orienté (ou une arborescence enracinée) ;
 - elles permettent ainsi de forcer la connectivité entre les drones tout en évitant les sous-tours non désirés.

Dans la construction des contraintes (8) et (9), nous utilisons la méthode du **big M**. En choisissant une constante M suffisamment grande mais raisonnable, on peut garantir que la condition reste toujours vérifiée dans l'implémentation pratique du modèle. On remarque également que, bien que z_i soit une variable continue, la contrainte (11) garantit que, dès lors que z_i est non nul, sa valeur doit être égale à 1. En effet, comme l'objectif est de maximiser la somme des z_i , le solveur aura toujours intérêt à les pousser vers leur valeur maximale dès que cela est autorisé par les contraintes.

Enfin, $x_{\min}^S$, $x_{\max}^S$, $y_{\min}^S$ et $y_{\max}^S$ représentent les bornes de la zone dans laquelle les drones peuvent être placés. Ces valeurs correspondent aux abscisses et ordonnées minimales et maximales des points de grille utilisés dans la méthode PLNE.

Cependant, en pratique, dès que le nombre de drones dépasse cinq, le temps de calcul devient considérable. Dans des contextes avec une contrainte de temps, il arrive même que le solveur ne parvienne pas à trouver une solution réalisable (non-convergence).

3 Travaux réalisés

3.1 Méthode combinant PLNE et PQNE

Pour tirer parti de la rapidité de la PLNE tout en profitant de la souplesse continue de la PQNE, nous proposons l'algorithme suivant, qui combine :

1. Une première phase de résolution discrète (PLNE) pour obtenir un placement initial.
2. La définition de PQNE autour de chaque position de drone trouvée à l'étape 1.
- Note :** Le warm-start est le fait de donner une solution réalisable en entrée du modèle.
3. Une phase finale de recalage dans ces petits volets locaux à l'aide d'un solveur quadratique (PQNE).

Algorithm 3: Hybride PLNE–PQNE avec perturbation locale

Entrées: Instance I ,
 Ensemble discret de positions P ,
 Nombre de drones p ,
 Rayon de perturbation $\delta > 0$,
Sortie: Solution continue ajustée S^*

```

// 1. Phase discrète (PLNE)
1  $S_{\text{disc}} \leftarrow \text{SOLVEPLNE}(I, P, p)$ ;
// 2. Construction des voisinages locaux
2 foreach  $s \in S_{\text{disc}}$  do
3    $N(s) \leftarrow \{x \in \mathbb{R}^2 : \|x - s\| \leq \delta\}$ ;
// 3. Création de l'instance quadratique restreinte
4 Construire  $I'$  en limitant l'espace de recherche à  $\bigcup_{s \in S_{\text{disc}}} N(s)$ , en imposant que chaque
   voisinage  $N(s)$  contienne exactement un drone;
// 4. Phase continue (PQNE) avec warm start
5  $S^* \leftarrow \text{SOLVEPQNE}(I', \text{warm\_start} = S_{\text{disc}}, P)$ ;
6 return  $S^*$ ;

```

Ceci revient, dans le modèle (PQNE) à remplacer :

$$\alpha_k \in [x_{\min}^S, x_{\max}^S], \quad \beta_k \in [y_{\min}^S, y_{\max}^S]$$

$$\text{par } \alpha_k \in [\alpha_k^{\text{WS}} - \delta, \alpha_k^{\text{WS}} + \delta], \quad \beta_k \in [\beta_k^{\text{WS}} - \delta, \beta_k^{\text{WS}} + \delta],$$

où $(\alpha_k^{\text{WS}}, \beta_k^{\text{WS}})$ est la position issue du warm start et δ le rayon de perturbation autorisé.

3.1.1 Valeur propre

Contrairement à la fonction objectif du PLNE décrite en section 2.2, nous proposons ici une approche complémentaire visant à maximiser la **deuxième plus petite valeur propre** (aussi appelée *valeur propre de Fiedler*) de la matrice de distances $\mathbf{W}$ entre drones activés.

Cette quantité, directement liée à la *connectivité algébrique* du graphe formé par les drones, constitue un indicateur mathématique de la **robustesse topologique du réseau** : plus cette valeur est élevée, plus le graphe est bien connecté, ce qui se traduit en pratique par une meilleure résilience du réseau face aux perturbations et une convergence plus rapide vers un état stable.

Ce critère a initialement été introduit dans des problèmes de localisation de capteurs [3], où l'objectif était d'assurer une couverture stable tout en maintenant la connectivité entre nœuds. Nous explorons ici la possibilité d'adapter ce critère à notre propre contexte, en l'intégrant comme objectif secondaire complémentaire à la maximisation du nombre d'unités couvertes dans le PLNE.

Dans cette optique, même si l'on accepte de sacrifier légèrement la couverture (par exemple, en plaçant les drones de manière plus centralisée plutôt que directement au-dessus des clients), le gain potentiel en temps de calcul lors de la phase continue (PQNE) peut s'avérer significatif. Cela est particulièrement vrai lorsque la solution de départ est mieux structurée, ce qui facilite la convergence vers un optimum local dans la phase quadratique.

Limites et exigences techniques : Cette approche requiert néanmoins des outils supplémentaires : la construction explicite de la matrice de distances $\mathbf{W}$ entre drones activés, ainsi qu'un solveur capable de traiter des problèmes d'optimisation incluant des contraintes semi-définies positives (SDP). Ce dernier point dépasse les capacités des solveurs classiques utilisés pour le PLNE, et justifie un traitement séparé de cette formulation dans le cadre expérimental.

3.2 Implémentation

Prise en main

Au cours de ce stage, je me suis d'abord pleinement familiarisé avec le code existant et compris en détail l'algorithme PLNE ainsi que l'algorithme glouton. J'ai ensuite implémenté l'algorithme 3 directement dans la fonction `solveGreedy` et expérimenté la faisabilité d'ajouter la valeur propre à la fonction objectif.

Débogage

Dans la section 2.3, la contrainte (9) n'était pas respectée lors de l'initialisation en « warm start » des solutions PLNE, pour deux raisons principales :

1. **Valeur de ε sous-estimée.** Lors de la modélisation, le paramètre ε choisi n'était pas assez grand : dans certains cas, la distance entre un drone et un client excédait ε , ce qui forçait $c_{ik} = 0$ et violait la contrainte, alors même que la solution restait valide. En particulier, dans le cas où les clients sont relativement regroupés, mais où certains points de la grille (non activés) se situent dans des zones très éloignées (par exemple, dans les coins du domaine), la contrainte peut être faussement violée, car la distance entre ces sites et les clients dépasse la valeur de ε choisie. Pour éviter ce problème, nous avons opté pour une définition plus robuste de ε , en le fixant à la distance maximale possible dans le cadre spatial considéré, c'est-à-dire la **longueur de la diagonale du domaine étudié** (souvent appelé “canvas”). Cette valeur garantit que, quelle que soit la position d'un site non activé, la contrainte reste mathématiquement toujours vérifiée dès lors que la variable de couverture associée vaut zéro.
2. **Erreur dans le calcul des distances.** À l'import des instances, la distance entre le client i et le site j a été calculée en deux dimensions, sans tenir compte de l'altitude, ce qui biaisait la matrice des distances et violait la contrainte (9). En fait R_{cov} contient l'altitude L .

Afin de faciliter la définition du warm start du modèle PQNE, la fonction `solveGreedy` a été modifiée pour renvoyer la matrice indicatrice c_{ik} dans ses résultats. Cette matrice $(c_{ik})_{i,k} \in \{0,1\}^{|C| \times |S|}$ est ensuite injectée en tant que warm start dans `solveQuadratic`, en affectant directement les variables binaires selon les valeurs de c_{ik} . Le warm start n'était parfois pas accepté par le solveur quand on ne donnait que la position des drones mais qu'en ajoutant les variables c_{ik} , il était plus souvent accepté.

```
1      for hId in 1:length(sortedPositions)
2          h = sortedPositions[hId]
3          for i in 1:instance.n
4              # set the coverage of client i by site hId
5              dist2 = (h[1] - instance.clientPositions[i, 1])^2 + (h[2] -
6                  instance.clientPositions[i, 2])^2 + instance.L^2
7              if dist2 <= instance.rCouv^2 + 1e-6
8                  set_start_value(dVariables[:c][i, hId], 1)
9              else
10                 set_start_value(dVariables[:c][i, hId], 0)
11             end
12             set_start_value(dVariables[:alpha][hId], h[1])
13             set_start_value(dVariables[:beta][hId], h[2])
14             @constraint(model, dVariables[:alpha][hId] >= h[1] - windowSize)
15             @constraint(model, dVariables[:alpha][hId] <= h[1] + windowSize)
16             @constraint(model, dVariables[:beta][hId] <= h[2] + windowSize)
17             @constraint(model, dVariables[:beta][hId] >= h[2] - windowSize)
18         end
19         if haskey(dVariables, :y)
20             set_start_value.(dVariables[:y], 0)
21             set_start_value.(dVariables[:y][sortedSiteIdx], 1)
22         end
23     end
```

3.3 Résultats numériques

Dans cette section, nous présentons les performances obtenues avec la nouvelle approche hybride (PLNE + PQNE) sur un ensemble de 18 instances, évaluées selon différents paramètres et différentes méthodes dans 1800 secondes. Ces instances sont réparties en deux groupes, correspondant aux tailles $n = 100$ et $n = 1000$, chacun contenant neuf tailles de grille différentes. Dans un premier temps, nous étudions l'évolution des résultats pour chaque instance en fonction de la taille de la fenêtre (l'étape 3 de l'algorithme 2). Ensuite, nous comparons les performances des quatre méthodes retenues — MTZ, greedy, gQuad et MTZQuad — afin d'évaluer leur efficacité respective en termes de couverture, de temps de résolution et de robustesse.

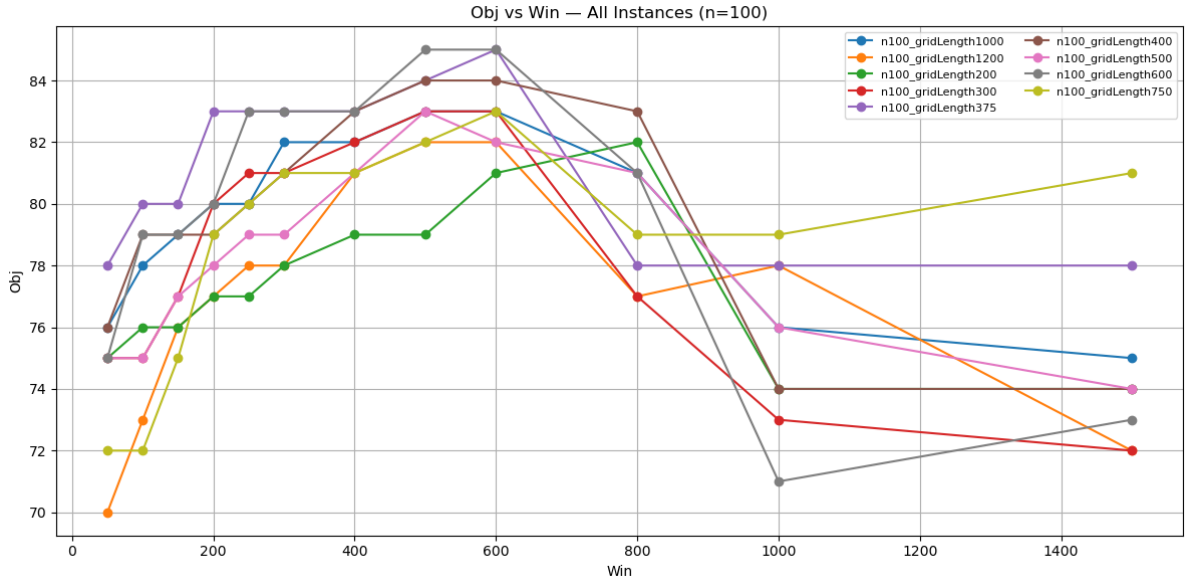


FIGURE 2 – Valeur d'objectif obtenues sur des instances $n=100$ en faisant varier la taille de la fenêtre. Chaque courbe correspond à une instance.

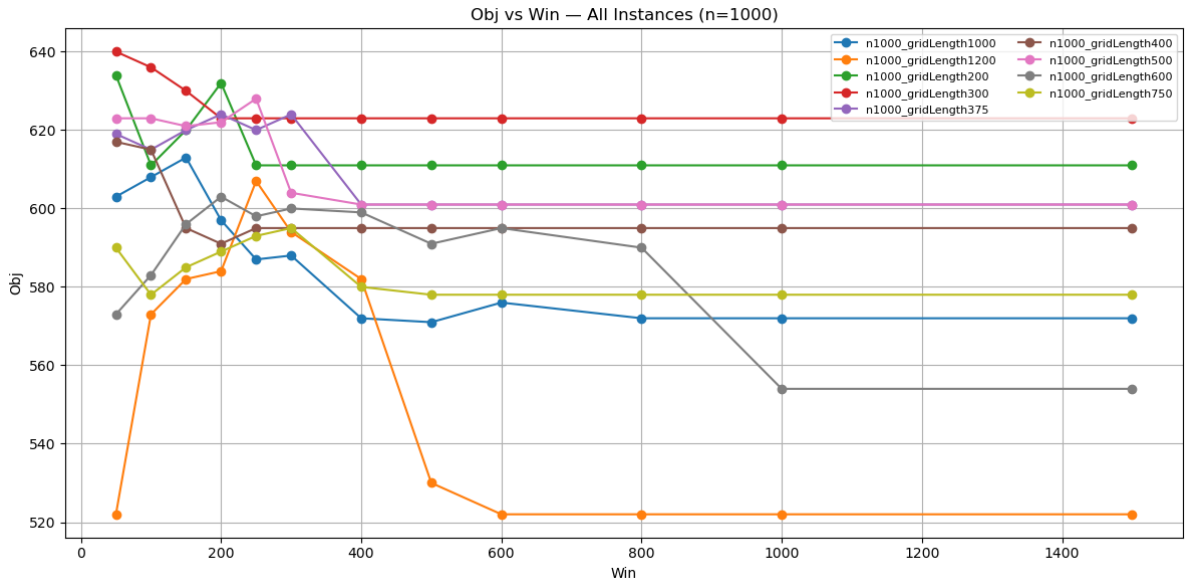


FIGURE 3 – Valeur d'objectif obtenues sur instances $n=1000$ en faisant varier la taille de la fenêtre. Chaque courbe correspond à une instance.

Les figures 2 et 3 représentent le nombre d’unités couvertes en fonction de la taille de la fenêtre. Chaque courbe correspond à une instance.

Dans la majorité des instances, la **valeur objective augmente avec la taille de la fenêtre**, puis se stabilise ou fluctue légèrement. Cela indique que l’effet de la taille de la fenêtre n’est **pas linéaire**. On observe généralement une *taille critique de la fenêtre* au-delà de laquelle les bénéfices deviennent marginaux.

Cas $n = 100$ (petite échelle) :

- Les valeurs objectives sont relativement faibles (environ 70–85), avec une variance faible.
- La solution converge rapidement, souvent dès les tailles de fenêtre moyennes.
- Les gaps sont presque toujours nuls : le solveur trouve aisément des solutions optimales ou quasi-optimales.
- Le temps de résolution est court.

Cas $n = 1000$ (grande échelle) :

- Les valeurs objectives sont plus élevées (souvent supérieures à 500), avec une variabilité plus marquée.
- Certaines instances (e.g. `gridLength1200`, `gridLength1500`) sont particulièrement sensibles à la taille de la fenêtre.
- Les gaps sont plus importants (souvent $> 0,2$), montrant la difficulté à atteindre l’optimalité dans le temps imparti.
- Le temps de résolution est significativement plus élevé (souvent > 5000 secondes).

Lorsque la taille de la fenêtre devient trop grande, le problème PQNE sous-jacent devient **plus difficile à résoudre**. Le solveur échoue souvent à améliorer la solution initiale transmise en *warmstart*, ce qui explique pourquoi la valeur objective reste constante ou n’évolue plus.

Critère	$n = 100$	$n = 1000$
Convergence	Rapide	Plus lente
Stabilité de la solution	Élevée	Moyenne à faible
Gap	Généralement nul	Parfois significatif ($> 0,2$)
Temps de résolution	Faible	Élevé (> 6000 s possible)
Sensibilité à la fenêtre	Faible	Élevée

Table 1 Comparaison des performances selon la taille du problème

Fait intéressant, dans le cas de la formulation PLNE+PQNE, nous avons observé que le nombre d’unités couvertes ne décroît pas systématiquement avec l’augmentation de la taille de la grille, contrairement à ce que l’on pourrait attendre. Deux raisons principales peuvent expliquer ce phénomène :

1. la distribution des clients est souvent non uniforme, certains étant fortement regroupés autour de points précis ;
2. la taille de la grille n’est pas toujours proportionnelle de manière optimale à la configuration spatiale du domaine étudié, ce qui peut conduire à ce que certaines grilles grossières capturent mieux les zones de densité élevée qu’une grille plus fine.

Plus on augmente la taille de la fenêtre, plus on s’approche du problème PQNE sans fenêtre qui n’est pas capable de converger.

Par ailleurs, pour $n = 1000$, nous constatons une plus grande instabilité du critère objectif selon la taille de la grille. Cela suggère que la solution initiale fournie par le warm start peut être significativement éloignée de l’optimum global dans les cas de grande taille.

Le tableau 2 présente une comparaison des différentes approches sous une contrainte de temps fixée à 1800 secondes. Les méthodes comparées sont les suivantes :

- **MTZ** : résolution classique du problème PLNE à l’aide de la méthode Branch&Bound, en utilisant la solution gloutonne comme solution initiale (warm start) ;
- **Greedy** : méthode gloutonne probabiliste (algorithme 2), où l’on conserve la meilleure solution obtenue parmi 20 simulations indépendantes ;
- **gQuad** : méthode hybride combinant PLNE et PQNE (algorithme 3), en utilisant la solution gloutonne comme point de départ ;
- **MTZQuad** : même principe que gQuad, mais la solution initiale est issue de la méthode MTZ.

Pour les méthodes gQuad et MTZQuad, la taille de la fenêtre de perturbation a été fixée, dans chaque cas, à la valeur correspondant à celle ayant produit le meilleur objectif lors des expérimentations précédentes.

n	η	Nombre d'unités couvertes				Gap final (%) du meilleur méthode	Temps (s)			
		MTZ	greedy	gQuad	MTZQuad		MTZ	greedy	gQuad	MTZQuad
100	200	74	74	82	82	0.0366	1589	94	-	-
	300	79	77	83	83	0	892	24	384	-
	375	80	78	85	85	0	215	10	1787	-
	400	75	74	84	84	0	106	7	171	180
	500	77	74	83	83	0	70	4	680	796
	600	74	71	85	83	0	59	3	534	262
	750	74	73	83	82	0.0361	26	2	-	1244
	1000	73	72	83	83	0	2	1	197	196
	1200	66	66	82	82	0	2	1	241	249
1000	200	637	629	634	652	0	-	416	-	1533
	300	631	631	640	637	0.241	-	64	-	-
	375	612	612	624	628	0.1	-	46	-	-
	400	597	595	615	615	0	-	29	-	-
	500	614	603	628	616	0.1744	866	31	-	-
	600	579	579	603	579	0.0644	-	19	-	-
	750	588	578	595	595	0	81	9	-	-
	1000	583	576	613	583	0.1362	12	7	-	-
	1200	528	525	607	600	0.1492	8	5	-	-

Table 2 Comparaison des méthodes

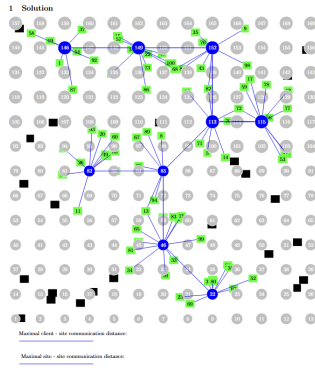
Il convient tout d'abord de noter que l'algorithme 3 apporte une amélioration significative de la fonction objectif par rapport au PLNE seul, et que ce gain devient d'autant plus marqué lorsque le nombre de clients augmente. Par ailleurs, les performances de gQuad et MTZQuad sont très proches en termes de qualité de solution, la principale différence résidant dans le temps de calcul.

Plus précisément, dans le cas $n = 100$, l'approche MTZQuad est pénalisée par le coût élevé de la phase Branch&Bound classique, surtout lorsque la grille est fine : le processus d'élagage devient alors particulièrement lent. En comparant les résultats obtenus par MTZ et Greedy selon différentes tailles de grille, on observe que le temps de résolution diminue avec l'augmentation de la taille de la grille, tandis que la valeur de la fonction objectif décroît également — un comportement cohérent avec nos attentes initiales.

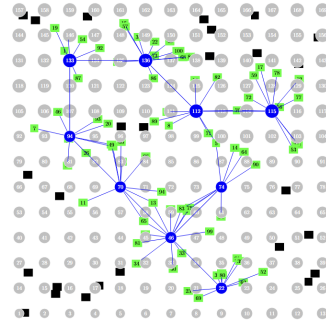
Enfin, d'un point de vue théorique, la méthode MTZQuad devrait fournir de meilleures solutions que gQuad (par exemple dans le cas $n = 1000$, grille de taille 200). Cependant, en pratique, le temps économisé par l'approche gloutonne permet à gQuad de consacrer davantage de ressources à la résolution du problème quadratique, ce qui se traduit par de meilleures performances sur des instances de grande taille.

Il convient toutefois de souligner que cette observation ne se vérifie pas nécessairement dans tous les cas, puisque la nature du problème résolu dépend fortement de la solution initiale : selon qu'elle provienne de Greedy ou de MTZ, les contraintes actives diffèrent, et les méthodes gQuad et MTZQuad peuvent alors résoudre deux problèmes d'optimisation légèrement distincts.

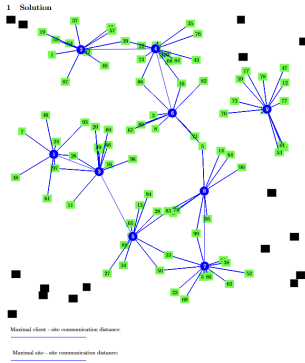
La figure 4 représente les solutions obtenues pour les 4 méthodes considérées. On peut noter que dans les deux solutions du dessous utilisant la PQNE, les drones ont été légèrement déplacés, permettant ainsi d'augmenter respectivement de 6 (à gauche) et 5 (à droite) le nombre de clients couverts.



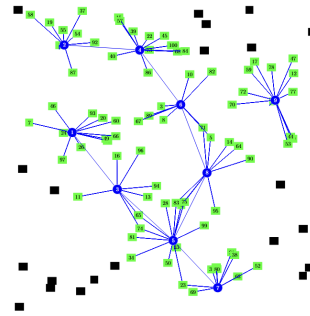
(a) MTZ, obj=77



(b) Greedy, obj=74



(c) MTZQuad, obj=83



(d) gQuad, obj=79

FIGURE 4 – Représentation visuelle finale de la solution pour un même exemple d'instance ($n=100$, $\eta=500$) selon les quatre méthodes comparées. Les figures (a) à (d) correspondent respectivement aux méthodes MTZ, greedy, gQuad et MTZQuad. À noter que, pour une meilleure équité de comparaison, les méthodes gQuad et MTZQuad utilisent la même taille de fenêtre, qui n'est pas nécessairement optimale.

Conclusion

Les Figures 5 et 6 présentent l'objet de chaque méthode en fonction de la taille de la grille (Temps limits = 1800s). Sur la base des résultats expérimentaux, plusieurs constats peuvent être formulés :

- Sous une contrainte de temps identique, les instances avec un nombre réduit de clients ($n = 100$) permettent généralement une plage de perturbation plus large autour de la solution initiale, ce qui laisse davantage de liberté d'ajustement dans la phase quadratique.
- La méthode hybride PLNE+PQNE améliore systématiquement la qualité des solutions, quel que soit le paramètre de discrétisation utilisé. En particulier, lorsque le nombre de clients est faible, cette approche tend à atténuer l'impact négatif lié à une grille trop grossière, ce qui se traduit par des résultats plus stables et plus performants.
- Le comportement des méthodes reste sensible à la structure de la grille : bien que l'algorithme glouton permette un bon compromis entre temps et qualité, il s'éloigne parfois de l'optimum global, notamment pour les grandes instances où la solution de départ joue un rôle clé dans la convergence.

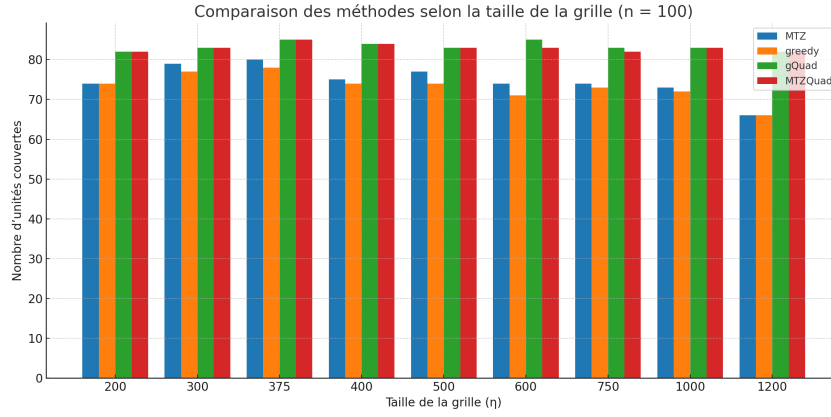


FIGURE 5

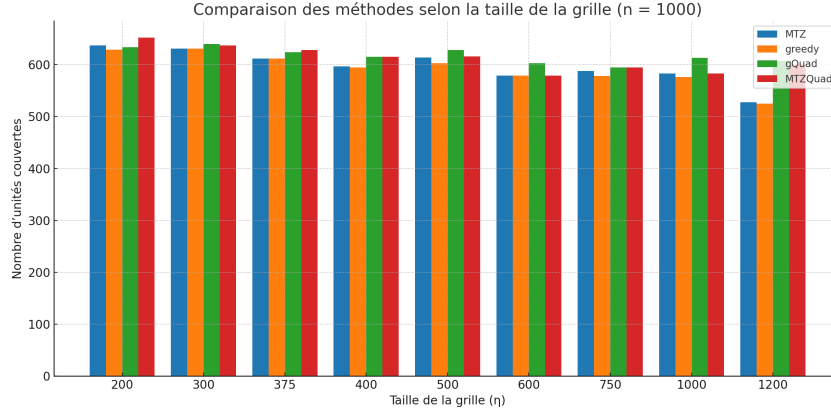


FIGURE 6

Ces observations ouvrent plusieurs perspectives de recherche. Il serait pertinent d'étendre l'étude à d'autres formulations de connexité que la version MTZ utilisée dans ce travail, telles que des schémas de communication alternatifs entre drones, ou encore l'ajout de capteurs relais pour améliorer la connectivité. Par ailleurs, afin d'accélérer la résolution des instances de grande taille via la méthode Branch&Bound, il conviendrait de concevoir de nouvelles familles d'inégalités de coupe, mieux adaptées à un nombre important de drones et plus efficaces que celles testées dans ce stage.

A L'exemple de l'instance

```
1 n = 1000
2 m = 49
3 rCouv = 1500
4 rCom = 1500
5 L = 3700
6 p = 1
7 d = Any[]
8 d_com = Any[]
9 clientPositions = [5902.12 4832.33; 3752.66 4136.95; 3439.26 5889.04; 3609.87
    3076.26; 5718.87 1065.93; 1210.17 672.65; 1460.1 3896.54; 368.32 2265.09;
    2232.86 416.88; 1436.25 3593.89; 5956.68 5321.27; 5489.6 463.77; 1648.66
    3012.81; 635.94 3180.28; 745.55 394.75; 2599.88 3537.54; 4428.09 429.23;
    762.81 5457.41; 4071.67 3013.61; 5128.89 2637.18; 1622.77 936.62; 3095.21
    4357.81; 3275.67 2574.12; 946.99 31.9; 3929.37 5167.44; 2812.2 576.51;
    4447.18 1751.44; 1933.81 1339.57; 1323.31 2622.05; 1773.87 663.06; 2952.57
    2991.64; 3262.42 5282.11; 5393.74 3449.61; 4909.28 5376.64; 5734.15 2335.49;
    490.22 917.96; 4760.46 2201.45; 2819.54 4246.5; 42.1 1968.05; 3005.12
    2950.76; 2154.2 5586.23; 848.29 5391.6; 1600.74 4592.65; 4286.47 2052.8;
    4997.56 2533.93; 4361.92 5158.74; 5078.34 911.94; 5022.51 3094.01; 3688.81
    678.09; 4400.86 3289.4; 782.18 1036.76; 4239.97 2850.37; 1878.68 3638.87;
    5401.41 2963.94; 4441.24 5141.88; 442.76 1020.94; 1992.28 585.47; 5702.47
    4682.25; 1195.37 5418.87; 2710.81 909.95; 5191.05 130.42; 630.23 1360.41;
    666.88 5174.03; 396.2 1509.76; 5082.88 2780.17; 2561.04 4669.43; 3454.4
    3799.26; 3492.76 1445.41; 1200.79 5679.7; 840.89 2687.17; 2445.68 526.92;
    5074.18 1656.07; 5797.11 2855.25; 783.41 2754.39; 4122.68 2185.49; 717.36
    188.85; 1951.64 821.87; 3731.9 4005.88; 1426.89 4535.26; 5695.31 1697.97;
    4752.74 3270.48; 2042.8 4849.43; 80.14 533.14; 4314.89 3326.96; 5647.92
    1940.69; 2911.95 5434.86; 3676.19 2502.03; 1118.95 471.47; 263.43 1183.53;
    593.22 3169.24; 431.56 3852.0; 5347.69 3471.31; 854.88 5968.37; 3118.16
    891.56; 2988.69 3240.65; 3915.38 4529.26; 752.15 101.38; 3279.03 1460.45;
    796.53 5654.1; 3292.93 5774.23; 5260.47 1405.97; 5732.35 3875.23; 3322.16
    569.02; 2690.78 2623.42; 861.56 532.19; 1040.86 2052.63; 2892.57 2246.53;
    5256.62 2350.06; 2269.78 2975.45; 2425.44 868.55; 547.08 5536.79; 5308.23
    4948.43; 826.05 169.96; 1974.84 5520.67; 4315.12 4995.2; 4701.5 2675.22;
    144.43 2270.84; 5241.02 193.53; 251.79 1742.88; 4081.9 818.92; 3643.39
    3183.02; 1629.54 1217.93; 1739.39 1335.35; 2937.19 196.86; 4700.12 4875.42;
    353.33 1438.84; 5952.7 1636.4; 4726.1 86.83; 1358.5 2093.44; 2127.18
    2587.42; 2261.33 3427.23; 298.77 3217.13; 691.66 3650.65; 92.87 3198.11;
    2531.48 562.87; 1440.35 4777.39; 2750.2 516.36; 2549.04 1978.3; 2052.38
    2229.78; 3483.92 848.85; 2611.41 4344.21; 2606.61 1974.19; 5418.4 794.89;
    4955.11 1024.45; 2021.04 2957.06; 280.74 3767.52; 4537.65 2540.32; 3516.17
    694.58; 224.36 75.54; 5020.18 1282.69; 3123.97 1820.56; 2378.09 4902.42;
    5029.73 1600.33; 5148.1 3687.86; 5859.86 5784.77; 4330.84 70.99; 4304.54
    4193.71; 5331.73 4937.45; 4628.19 5511.01; 3326.93 3530.3; 1102.14 5548.9;
    2203.97 4426.39; 5209.76 2301.39; 5164.26 3817.76; 5950.45 4228.97; 1720.71
    4275.52; 2211.34 1310.9; 749.36 2614.1; 1741.92 751.27; 2854.88 3790.83;
    5088.63 1902.97; 360.35 2023.57; 4476.45 5885.26; 5791.7 5603.3; 5038.41
    2368.71; 882.24 3932.02; 2078.53 196.06; 3807.95 1806.79; 3632.68 3172.97;
    4251.2 1431.97; 2833.76 4268.2; 2208.36 1397.97; 2034.86 3215.81; 3334.31
    1237.5; 1297.44 607.67; 3043.41 2995.57; 2371.95 1839.45; 1968.8 2191.58;
    3126.87 2151.85; 1279.93 4114.28; 1922.9 4526.91; 3778.52 2479.49; 3066.59
    4287.71; 21.09 5277.24; 556.76 3996.77; 1107.8 3779.15; 3753.63 2147.82;
    2278.34 1777.71; 4073.47 1850.92; 1332.72 3295.9; 5535.33 1169.22; 4440.28
    4747.96; 92.61 1664.97; 1737.57 1663.44; 5137.29 3913.51; 2938.73 3731.01;
    5535.49 284.32; 81.01 4244.81; 5177.71 5571.38; 2423.36 181.5; 1721.86
    440.26; 5896.15 3459.33; 41.15 3140.55; 5084.93 3977.42; 3957.72 4523.53;
    2359.16 4536.27; 1840.84 316.53; 128.28 196.62; 2648.97 1845.69; 520.89
    284.33; 1960.39 2541.11; 4828.09 4111.22; 5970.29 2663.56; 5006.03 282.77;
    4485.4 2093.33; 5251.93 1417.8; 3721.24 5418.02; 2897.96 4175.19; 651.19
    3738.35; 2696.57 4779.54; 616.69 4081.68; 594.07 2569.1; 3571.6 213.12;
    5788.3 3646.04; 1583.07 96.15; 1063.55 843.76; 3342.56 5633.82; 4428.85
```

190.39; 1667.15 5694.15; 735.93 663.28; 2588.76 3038.0; 4584.65 3346.88;
 2306.88 325.83; 4232.63 3965.19; 3483.12 587.65; 707.51 2819.1; 4944.68
 4910.61; 3816.83 776.01; 2977.97 3679.85; 2428.71 661.87; 3295.19 5835.9;
 3324.07 321.62; 3282.02 3015.45; 4149.24 5812.47; 480.83 2681.78; 1338.24
 3053.4; 226.36 575.04; 5625.8 4260.45; 2619.61 2497.08; 2279.14 4293.87;
 447.67 3351.98; 3196.91 2855.14; 3390.3 4411.44; 3049.38 4806.54; 97.09
 3716.35; 5634.06 5807.03; 1819.57 2142.5; 2304.08 2048.83; 2285.33 4473.1;
 2176.23 5865.54; 1985.97 2668.1; 2505.78 4329.82; 5220.25 5877.11; 2384.6
 5624.39; 1314.35 4365.09; 2396.14 5431.86; 454.07 1146.1; 37.55 4999.67;
 3838.25 4017.34; 904.23 12.56; 1479.67 3346.24; 3432.82 3258.18; 4856.76
 4126.65; 4251.91 1175.36; 1082.42 3777.86; 1996.98 4026.07; 4656.53 4812.45;
 3109.95 2193.05; 646.52 2943.73; 2772.59 4559.96; 469.88 3230.96; 1393.6
 643.27; 2872.64 5806.94; 3129.5 3632.48; 1031.58 3973.25; 4112.66 4865.5;
 5391.18 1269.59; 2515.5 1402.31; 1782.32 5224.44; 244.88 200.06; 3311.29
 3425.07; 4732.98 3937.55; 301.85 3416.76; 3685.11 2688.0; 289.18 3036.82;
 5137.82 5521.25; 4253.33 1135.6; 4401.16 3374.38; 4313.36 1182.3; 5050.31
 377.23; 3807.11 5109.38; 3017.86 1736.15; 5778.97 4817.35; 5829.67 4984.83;
 1405.12 95.82; 1582.9 2512.35; 4343.86 4170.2; 5736.1 1902.23; 2742.52
 1070.58; 4385.7 4573.49; 5350.46 34.06; 4973.83 374.02; 791.0 4066.03;
 1855.15 3363.12; 532.29 5322.9; 4484.98 3767.98; 4762.44 2709.29; 2569.64
 3029.72; 1698.73 1983.31; 678.94 3284.92; 5606.07 912.53; 3915.45 3501.64;
 1770.99 3262.01; 4752.65 4148.83; 5411.88 3627.72; 4623.92 2712.87; 1895.55
 5283.04; 5.59 2363.2; 2911.75 2880.83; 4073.62 3076.83; 4245.16 677.42;
 4691.18 1996.03; 3258.65 5378.33; 4206.46 4645.55; 3397.65 3269.55; 5689.34
 1701.56; 432.66 992.41; 2351.2 1076.72; 2111.57 5759.89; 5769.37 3070.71;
 32.15 2065.85; 388.66 2658.3; 2973.18 781.69; 3362.97 3785.73; 5294.37
 1516.93; 4469.94 2146.12; 5144.96 775.73; 5197.09 807.68; 428.35 3882.39;
 5732.8 2741.15; 4292.74 5643.11; 5250.76 1629.41; 5815.86 1276.22; 953.51
 4435.64; 1537.41 4394.02; 4490.5 2146.7; 5660.96 463.5; 5416.96 2232.56;
 3363.53 5644.9; 5123.33 863.32; 5500.37 3767.87; 5784.97 569.4; 2892.55
 3474.0; 1219.88 5928.94; 1006.59 1705.58; 3127.53 755.08; 2453.36 5674.75;
 2048.67 746.57; 5596.25 2496.96; 5727.59 2381.99; 2272.49 3918.61; 369.69
 5484.27; 415.7 3825.97; 950.52 5466.38; 3164.28 5562.79; 4007.66 5045.52;
 1628.72 56.61; 3041.56 3160.77; 5725.16 5078.25; 129.72 1402.79; 5671.61
 5605.14; 3109.84 536.59; 1001.02 3891.33; 812.1 47.05; 4242.58 1887.65;
 746.79 157.07; 5370.36 496.69; 3938.61 5022.08; 1879.25 1852.62; 4513.45
 3269.85; 2414.51 1563.21; 3564.08 1203.99; 4391.85 1814.79; 3670.69 1963.61;
 2081.57 3374.68; 1580.75 3303.34; 1676.99 821.74; 325.2 384.82; 557.0
 2536.2; 4335.3 889.47; 4647.47 5025.17; 165.07 3260.83; 2753.77 1417.82;
 2260.01 4191.24; 3843.85 5509.84; 1955.68 1025.37; 459.12 1068.53; 4525.19
 355.76; 1140.38 2996.42; 5122.88 2660.68; 4090.43 996.57; 5631.88 5007.29;
 2099.94 5514.18; 2640.77 3177.16; 4532.38 5125.62; 4789.43 1835.08; 3341.79
 1831.54; 4418.8 5679.92; 12.78 2127.69; 4066.11 4695.47; 5010.41 2416.89;
 5473.84 5948.41; 1855.82 3302.74; 2745.99 5292.55; 3796.43 5578.07; 138.0
 2656.14; 3179.27 5378.89; 3995.35 5933.92; 400.6 800.22; 2588.23 1903.38;
 2768.75 1848.93; 5707.01 3520.58; 970.58 5681.15; 317.07 4298.24; 3043.4
 2839.54; 4355.99 1138.67; 5800.6 4260.45; 1550.92 5634.99; 4273.21 749.68;
 1583.99 5775.7; 4823.71 5678.35; 3179.04 1386.99; 3651.86 4881.8; 3361.19
 3463.88; 122.0 5369.08; 2605.01 1471.44; 4738.33 5598.73; 3042.16 3140.05;
 1891.21 5769.58; 1003.12 1826.2; 5572.19 25.73; 934.91 2109.56; 3573.9
 2351.83; 5411.62 2044.26; 1001.49 4084.22; 1427.64 5051.75; 5892.8 5719.72;
 2134.83 5317.06; 1920.78 2792.72; 1678.68 4314.22; 5635.86 2198.35; 4157.51
 2506.15; 4656.32 4446.52; 265.95 1768.11; 3788.71 1714.12; 4571.47 4181.82;
 69.37 3158.39; 1552.75 2005.72; 4939.26 818.98; 347.66 1845.47; 5788.47
 1085.14; 249.12 716.19; 3681.98 1395.33; 1217.41 5496.22; 3796.51 2543.41;
 1853.02 1496.38; 4246.7 1824.15; 274.47 2217.62; 1386.99 4581.42; 3146.98
 4457.29; 1019.21 1842.73; 676.48 3034.59; 2829.91 1288.3; 4225.46 731.45;
 5578.0 5848.15; 4209.01 2369.07; 3513.02 3557.26; 3848.96 3580.33; 1273.62
 3289.77; 5770.58 4531.44; 1748.87 2288.47; 4186.84 64.3; 926.61 2026.6;
 4025.09 3338.26; 2853.09 2125.41; 87.35 4801.53; 3079.29 2046.66; 1816.76
 4482.09; 1294.05 270.63; 3398.5 1266.58; 4103.49 2136.43; 1902.99 1369.73;
 2503.87 5395.8; 5259.46 268.71; 3732.79 1228.61; 5927.22 3862.66; 4913.4
 5545.19; 2147.6 2275.25; 1673.46 3456.92; 5153.04 3953.26; 4459.93 4915.6;

317.22 5195.78; 516.45 5251.95; 1201.64 4391.87; 3820.11 961.22; 2902.54
4537.96; 1568.41 3861.82; 5489.11 3427.75; 1261.47 4219.6; 4639.13 5780.2;
72.93 1104.62; 1017.93 5467.34; 5205.92 1366.46; 4280.99 2973.71; 3453.35
1119.25; 4891.91 992.05; 738.95 441.0; 5035.11 4022.79; 604.89 2192.83;
1657.43 2055.05; 3901.66 4323.23; 2446.32 4804.94; 5290.52 2637.92; 3534.7
4113.4; 5154.62 4850.78; 159.99 4071.93; 238.14 4050.21; 2213.65 1189.9;
2160.96 1791.8; 2579.55 4382.67; 5537.59 4702.61; 1912.14 3850.52; 2910.12
956.85; 5996.0 5010.04; 152.48 3107.77; 829.7 1275.97; 5632.1 5797.74; 4.81
3265.59; 2976.23 233.92; 5549.3 5252.34; 4069.75 1367.76; 2847.9 1265.98;
3133.7 2404.22; 2379.54 3175.74; 3916.47 3153.23; 4853.88 4119.19; 3873.72
862.85; 4042.58 2520.98; 5978.8 940.83; 3218.84 2932.61; 3776.34 2932.27;
2738.24 1662.22; 2980.9 188.65; 2830.81 4799.26; 2294.53 3588.58; 541.1
1542.58; 3538.33 964.02; 2172.6 48.47; 2164.36 4614.44; 4322.02 5759.71;
5805.34 4306.13; 3854.44 4613.46; 3701.52 3509.54; 5200.82 3721.44; 1511.51
2920.43; 589.46 3508.23; 3709.64 3610.55; 1149.06 852.17; 5699.96 4041.36;
2890.78 1983.94; 5098.71 1825.34; 259.8 4476.16; 2253.16 1003.92; 5409.83
858.4; 2262.9 5953.03; 469.36 532.96; 5667.95 3593.01; 5529.84 121.58;
2703.54 452.65; 2412.31 1393.17; 3748.05 588.89; 4684.65 3377.92; 490.36
1749.28; 5881.91 4631.67; 4384.12 5203.36; 1441.13 439.05; 1335.74 3265.35;
2475.17 3011.59; 2189.88 1095.14; 5895.62 1658.05; 948.42 1481.02; 3985.53
49.77; 4447.35 1016.97; 1738.66 4610.25; 1103.87 2659.06; 4761.09 4963.04;
2494.12 3499.01; 1838.78 2661.29; 5862.14 4008.15; 4673.9 1128.97; 2441.39
2180.86; 3937.47 1427.94; 857.9 4105.3; 1666.51 4519.08; 3732.49 863.71;
4985.94 3877.03; 3754.36 4158.29; 5446.81 329.28; 2327.73 4483.06; 1453.21
3219.65; 4151.61 3153.24; 275.52 5133.3; 2251.76 1088.89; 81.82 4705.3;
1442.07 5263.42; 4978.77 2935.54; 5938.49 79.9; 5.66 2179.3; 3527.41
1677.59; 5138.25 3131.71; 2489.54 1238.21; 3030.44 68.01; 789.87 2930.65;
26.59 2739.37; 890.06 4885.67; 1010.82 1243.38; 2612.84 1430.29; 4557.0
308.61; 4955.4 4135.75; 2657.12 4837.62; 605.66 3023.88; 3068.34 3549.64;
5955.91 4227.48; 883.77 714.52; 882.67 97.76; 2252.21 5313.8; 1949.65
4677.16; 1047.79 1649.21; 409.51 3521.74; 198.46 5287.35; 4998.28 59.07;
3264.35 1801.11; 5697.97 4183.05; 5429.45 1141.66; 2908.65 1059.3; 4547.15
1882.34; 2136.46 856.6; 3997.33 3748.67; 3729.38 3971.84; 5376.89 4043.14;
192.81 5178.91; 4652.57 70.74; 1764.04 4799.14; 2979.94 1713.57; 5835.7
687.9; 2950.32 152.37; 845.8 3682.23; 1050.69 4213.34; 4254.46 1653.32;
1585.17 238.19; 286.74 4725.79; 2609.51 3045.86; 2099.71 988.43; 5461.97
3282.09; 5929.33 1568.64; 1523.44 534.73; 3464.79 5487.31; 2807.4 1975.64;
1220.53 48.9; 5807.43 4842.24; 5440.3 198.31; 4731.83 2640.41; 1451.67
4416.79; 1405.76 5501.35; 3125.3 4589.62; 2936.71 3913.2; 2198.96 1649.28;
442.58 5966.75; 3851.4 2162.99; 551.71 476.41; 5888.5 3016.85; 1128.12
1690.92; 5181.98 3948.92; 2137.78 230.32; 901.66 2790.17; 689.49 1935.61;
334.27 2519.27; 3362.84 5404.51; 4075.97 2322.37; 3440.93 5565.46; 1782.31
3776.66; 4663.48 365.65; 4682.52 4244.24; 1175.68 5245.59; 782.16 1092.22;
2938.81 3784.19; 91.42 5352.14; 4449.83 5812.74; 2816.97 2760.97; 5913.4
2859.31; 1288.86 2121.09; 3130.33 5186.19; 1492.02 5858.27; 5292.83 2775.0;
4412.49 5308.92; 5166.62 228.2; 2821.88 1335.47; 4435.6 1414.66; 3963.05
910.93; 2999.04 2657.72; 3970.71 2440.7; 2089.07 5150.05; 1367.84 228.27;
338.6 1267.16; 1385.25 4403.41; 279.2 2566.83; 1996.6 3367.25; 1906.64
4797.5; 915.89 890.2; 2145.15 5418.83; 3811.37 234.65; 1181.88 3920.19;
2535.09 492.19; 5576.63 3447.42; 2151.29 553.7; 538.14 4180.42; 881.66
3237.88; 2469.01 5678.0; 72.52 1707.06; 4787.52 1368.73; 452.69 3874.75;
5318.19 3997.55; 5375.38 4051.99; 3391.28 190.43; 877.59 3345.44; 2246.18
1367.98; 1041.54 5531.04; 4245.93 2757.77; 4085.74 874.87; 5298.03 5447.8;
4102.39 2389.97; 1301.24 5034.27; 2152.01 1748.11; 3082.16 482.02; 4546.3
4482.57; 2281.9 2957.27; 2739.07 4097.25; 4292.31 3333.32; 969.76 4318.0;
706.48 15.76; 787.2 3891.57; 1770.46 2797.12; 2544.92 5712.22; 660.88
3258.7; 5244.99 3915.1; 5660.17 2607.73; 1341.36 1041.76; 4001.92 2552.82;
3705.69 4694.09; 371.75 1888.03; 4853.11 4016.8; 727.07 2019.02; 4158.01
4546.59; 1413.67 348.13; 4090.75 5701.66; 3799.54 1702.64; 3683.7 5472.94;
3166.06 5128.7; 4645.39 801.45; 4038.92 4403.93; 5959.49 2175.73; 1054.48
1410.18; 4620.35 4898.21; 994.15 1334.23; 4271.56 957.57; 4615.03 5049.45;
3213.48 4039.95; 1128.83 4537.92; 3045.29 723.11; 11.46 4344.24; 561.77
5531.6; 5364.41 1535.01; 3372.08 4208.03; 3601.87 1885.81; 5968.16 154.78;

```

1095.83 68.67; 4884.52 5739.46; 970.91 3974.87; 3373.99 5663.97; 2536.34
5124.86; 711.66 4419.52; 617.35 247.48; 5353.87 407.61; 3858.27 4863.42;
4868.6 743.09; 5105.48 633.7; 1450.95 1569.64; 3206.6 4177.84; 4046.26
683.54; 4153.48 3746.41; 1652.76 3064.34; 4175.83 3597.2; 1931.56 1258.45;
5315.96 2485.78; 4861.75 5039.84; 4748.84 5750.16; 188.39 1750.39; 4637.29
65.51; 2872.25 2713.75; 3426.0 2180.59; 3684.01 1703.95; 2662.12 2283.41;
5213.91 1223.34; 2848.11 3689.04; 3452.65 2700.91; 1182.77 4827.85; 4063.26
4099.13; 3409.75 127.23; 4735.17 4874.95; 3565.91 1787.72; 4920.78 2890.4;
3496.73 1909.7; 1810.08 2241.3; 1169.26 4365.69; 2493.52 254.89; 5495.1
792.77; 1664.6 279.69; 5842.84 4840.16; 466.36 5053.17; 3290.06 2427.58;
5746.81 3491.87; 427.96 1338.4; 4183.46 4767.14; 5129.45 771.79; 1977.99
4301.93; 3575.31 2833.64; 1087.02 1453.53; 5377.08 4464.21; 2913.51 5865.37;
2478.9 2621.03; 543.71 3572.78; 5975.79 3624.23; 3315.94 4288.6; 868.98
2469.92; 5530.35 980.69; 5429.9 446.31; 2628.12 1666.11; 706.87 2389.37;
3521.37 5956.09; 3495.82 4550.35; 2726.14 1433.29; 3981.25 5866.11; 3690.99
1283.3; 3358.46 2657.23; 3079.57 1867.51; 4280.74 1541.28; 4974.66 806.74;
426.54 133.38; 94.01 4607.3; 1466.37 3857.69; 4999.7 664.93; 1356.51
4978.25; 4560.38 5045.9; 5451.9 5702.84; 4371.44 743.28; 92.92 2896.28;
4972.69 2328.81; 1461.53 263.23; 5375.34 4154.49; 688.71 818.46; 4291.23
5278.82; 4489.55 3152.41; 5644.37 2703.54; 1594.99 963.15; 5882.33 3961.09;
193.37 972.71; 1792.18 94.56; 4937.42 5813.55; 3045.61 4920.92; 5903.8
1219.12; 1625.33 1664.73; 482.66 433.41; 336.22 5946.03; 3770.27 5165.73;
643.08 4908.47; 4530.2 5971.76; 2963.1 5423.45; 1106.94 2542.45; 1145.82
426.78; 3624.53 5546.26; 4796.81 2492.01; 1244.14 1877.13; 3035.23 1143.54;
5185.06 4865.17; 3915.66 716.11; 3963.27 4389.26; 4497.04 1408.46; 4305.73
301.42; 3345.46 949.43; 4400.46 815.26; 893.53 2032.58; 865.98 4369.74;
4011.04 2207.67; 3562.38 3221.17; 946.01 1294.89; 1058.53 1382.37; 104.83
2170.73; 1673.3 2514.48; 4470.35 262.97; 5810.38 5293.31; 850.6 4859.37;
1325.88 4228.51; 1630.65 710.93; 1803.9 5169.26; 680.44 3164.01; 1774.69
5485.68; 2983.76 473.34; 641.11 4809.38; 3420.2 2442.45; 4815.16 650.88;
1180.68 3236.22; 5497.93 5208.48; 4955.3 712.11; 5605.15 3578.84; 3171.19
1518.07; 1513.88 3641.88; 957.07 147.66; 1394.34 5292.79; 58.41 1046.03;
4288.81 3113.21; 4997.08 4402.95; 5375.32 642.81; 4455.01 1832.11; 2418.5
4347.81; 5396.14 4554.76; 4973.91 5846.3; 3306.32 572.28; 299.67 325.13;
4833.89 2699.34; 2067.11 4740.95; 5138.67 4684.74; 2346.69 3683.37; 2986.74
2482.09; 4125.72 530.74; 2595.34 5356.06; 5461.93 3128.95; 4119.92 1541.36;
4587.79 2991.36; 2035.62 5757.18; 3773.74 576.32; 4098.03 5244.84; 4316.84
98.56; 5531.97 1727.45; 2213.56 4512.62; 2067.93 3367.79; 1581.31 13.04;
5123.07 1342.22; 2046.44 4979.64; 3567.08 2633.43; 932.75 1754.83; 765.93
1215.21; 3920.1 3847.9; 4301.83 2175.36; 1283.27 5643.32; 1553.28 4906.59;
5424.41 5367.68; 654.59 182.42; 4409.55 9.12; 4651.45 4777.42; 5101.75
1307.26; 2074.11 2404.91; 4151.42 5462.74; 2076.97 1318.53; 5735.86 5858.41;
1210.25 3817.73; 3180.26 1364.57; 330.5 565.99; 518.44 4320.42; 5796.84
4396.39; 5933.33 2856.42]

```

10

```

11 sitesPositions = [0.0 0.0; 1000.0 0.0; 2000.0 0.0; 3000.0 0.0; 4000.0 0.0;
5000.0 0.0; 6000.0 0.0; 0.0 1000.0; 1000.0 1000.0; 2000.0 1000.0; 3000.0
1000.0; 4000.0 1000.0; 5000.0 1000.0; 6000.0 1000.0; 0.0 2000.0; 1000.0
2000.0; 2000.0 2000.0; 3000.0 2000.0; 4000.0 2000.0; 5000.0 2000.0; 6000.0
2000.0; 0.0 3000.0; 1000.0 3000.0; 2000.0 3000.0; 3000.0 3000.0; 4000.0
3000.0; 5000.0 3000.0; 6000.0 3000.0; 0.0 4000.0; 1000.0 4000.0; 2000.0
4000.0; 3000.0 4000.0; 4000.0 4000.0; 5000.0 4000.0; 6000.0 4000.0; 0.0
5000.0; 1000.0 5000.0; 2000.0 5000.0; 3000.0 5000.0; 4000.0 5000.0; 5000.0
5000.0; 6000.0 5000.0; 0.0 6000.0; 1000.0 6000.0; 2000.0 6000.0; 3000.0
6000.0; 4000.0 6000.0; 5000.0 6000.0; 6000.0 6000.0]

```

B Les code essentiels

```

1 function getQuadraticModel(instance::Instance; time_limit::Int=-1, isRelaxation
::Bool=false, threads::Int=-1, useCallback::Bool=false, maxCutsCount::Int
=10, connexity::Symbol=:QuadMTZ)

```

2

```

3   xMin = minimum(instance.clientPositions[:, 1])
4   xMax = maximum(instance.clientPositions[:, 1])
5   yMin = minimum(instance.clientPositions[:, 2])
6   yMax = maximum(instance.clientPositions[:, 2])
7
8
9   xMin2= minimum(instance.sitesPositions[:,1])
10  xMax2= maximum(instance.sitesPositions[:,1])
11  yMin2= minimum(instance.sitesPositions[:,2])
12  yMax2= maximum(instance.sitesPositions[:,2])
13  @show dFurthestHAPS(instance::Instance,xMin2, xMax2, yMin2, yMax2)
14  delta = sqrt((instance.rCouv^2-instance.L^2)/2)
15  M = (xMax-xMin-2*delta)^2+(yMax-yMin-2*delta)^2###why??
16
17  ### Model declaration
18  model = Model(CPLEX.Optimizer)
19  #set_silent(model)
20  if time_limit != -1 && time_limit > 5
21      set_optimizer_attribute(model, "CPX_PARAM_TILIM", time_limit)
22  end
23
24  if threads != -1
25      MOI.set(model, MOI.NumberOfThreads(), threads)
26  end
27
28  ### Variables
29  dVariables = Dict{Symbol, Any}()
30
31  # 1 iff client i is covered by HAPS k
32  @variable(model, c[1:instance.n, 1:instance.p], Bin)
33  dVariables[:c] = c
34
35  # 1 iff (j,j1) is a connection
36  @variable(model, xp[j in 1:instance.p, j1 in 1:instance.p ; j1!=j], Bin)
37  dVariables[:xp] = xp
38
39  if isRelaxation
40      JuMP.relax_integrality(model)
41  end
42
43  # 1 iff client i is covered
44  @variable(model, 0 <= z[1:instance.n] <= 1)
45  dVariables[:z] = z
46
47  # Coordinates of the HAPS, to be real numbers
48  @variable(model, xMax2 >= alpha[k in 1:instance.p] >= xMin2)
49  # @variable(model, xMax-delta >= alpha[k in 1:instance.p] >= xMin+delta)
50  dVariables[:alpha] = alpha
51
52  @variable(model, yMax2 >= beta[k in 1:instance.p] >= yMin2)
53  # @variable(model, yMax-delta >= beta[k in 1:instance.p] >= yMin+delta)
54  dVariables[:beta] = beta
55
56  ### Objective
57  @objective(model, Max, sum(z[i] for i in 1:instance.n))
58
59  ### Constraints
60
61  # xp_k,k2 = 0 if the distance between the HAPS is > rCom
62  # Contrainte a remettre
63  @constraint(model, [k in 1:instance.p, k2 in 1:instance.p; k != k2], (alpha
    [k] - alpha[k2])^2 + (beta[k]-beta[k2])^2 <= instance.rCom^2 * xp[k, k2]
    + (M+1000) * (1-xp[k, k2]))

```



```

64
65 # c_i,k = 0 if the distance between the HAPS and the client is > rCouv
66 # @constraint(model, [i in 1:instance.n, k in 1:instance.p], (alpha[k] -
    instance.clientPositions[i, 1])^2 + (beta[k]-instance.clientPositions[i,
        2])^2 + instance.L^2 <= instance.rCouv^2 * c[i, k] + dFurthestHAPS(
            instance, i, xMin, xMax, yMin, yMax, delta) * (1-c[i, k]))
67 epsilon=dFurthestHAPS(instance::Instance,xMin2, xMax2, yMin2, yMax2)
68 @constraint(model, [i in 1:instance.n, k in 1:instance.p], (alpha[k] -
    instance.clientPositions[i, 1])^2 + (beta[k]-instance.clientPositions[i,
        2])^2 <=(instance.rCouv^2-instance.L^2)* c[i, k] +epsilon* (1-c[i, k]))
69
70 # Client i is covered if at least one HAPS covers it
71 @constraint(model, [i in 1:instance.n], z[i] <= sum(c[i, k] for k in 1:
    instance.p))
72
73 # There is exactly p-1 variables xp = 1
74 @constraint(model, sum(xp[k, k2] for k in 1:instance.p for k2 in 1:instance
    .p if k != k2) == instance.p-1)###not exactly?
75
76 # A site does not have more than one input link
77 @constraint(model, [j in 1:instance.p], sum(xp[j,j1] for j1 in 1:instance
    .p if j1 != j) <= 1)
78
79 if connexity == :QuadMTZ
80
81     # Index of site j in the connected arborescence
82     @variable(model, instance.p >= tp[j in 1:instance.p] >= 0)
83     dVariables[:tp] = tp
84
85     # The order of the nodes is consistent with the arborescence
86     @constraint(model, [j in 1:instance.p, j1 in 1:instance.p ; j1!=j],
        tp[j1] >= tp[j] + 1 - instance.p*(1-xp[j,j1]))
87
88 end
89
90 if connexity == :QuadMF3hh
91
92     # 1 iff the arc (k1, k2) is on the path from the root to HAPS number k
        (j1 = 0 corresponds to the root)
93     @variable(model, 0 <= f[k in 2:instance.p, k1 in 1:instance.p, k2 in 1:
        instance.p; k1 != k2 && k1 != k] <= 1)
94     dVariables[:f] = f
95
96     # No flow on (k1, k2) if the HAPS are not deployed or out of reach
97     @constraint(model, [k in 2:instance.p, k1 in 1:instance.p, k2 in 1:
        instance.p; k1 != k && k1 != k2], f[k, k1, k2] <= xp[k1, k2])
98
99     # Flow conservation (source and sink)
100     # The flow of k reaching k is equal to that of flow k leaving 1
101     @constraint(model, [k in 2:instance.p], sum(f[k, k1, k] for k1 in 1:
        instance.p if k1 != k) == sum(f[k, 1, k2] for k2 in 1:instance.p if
        k2 != 1))
102
103     @constraint(model, [k in 2:instance.p, k1 in 1:instance.p; k1 != 1 &&
        k1 != k], sum(f[k, k1, k2] for k2 in 1:instance.p if k2 != k1) ==
        sum(f[k, k2, k1] for k2 in 1:instance.p if k2 != k1 && k2 != k ))
104
105 end
106
107 # Symmetry breaking
108 #@constraint(model, [k in 1:instance.p, k2 in k+1:instance.p], (yMax-yMin)
    * alpha[k] + beta[k] <= (yMax-yMin) * alpha[k2] + beta[k2])
109

```

```

110 # Test : non-optimal symmetry breaking constraints
111 #@constraint(model, [k in 1:instance.p, k2 in k+1:instance.p], alpha[k2] +
    beta[k2] >= alpha[k] + beta[k] + 3000)##avoid symmetry solution
112
113 # If a callback is used to generate clique inequalities
114 if useCallback && !isRelaxation
115
116     MOI.set(model, MOI.NumberOfThreads(), 1)
117
118     cutsCount = 0
119     lastNodeId = -1
120
121     if size(instance.areClientsClose, 1) == 0 # areClientsClose[i, j] is
        true if client i and j can be covered by the same HAPS
122         computeCloseClients!(instance)
123     end
124
125     # Callback function
126     function callbackClique(cb_data::CPLEX.CallbackContext, context_id::
        Clong)
127
128         #isInteger = isIntegerPoint(cb_data, context_id)
129         isFractional = context_id == CPX_CALLBACKCONTEXT_RELAXATION
130
131         valueP = Ref{Clong}()
132         ret = CPXcallbackgetinfolong(cb_data, CPXCALLBACKINFO_NODEUID,
            valueP)
133
134         if lastNodeId != valueP[]
135             lastNodeId = valueP[]
136             cutsCount = 0
137         end
138
139         # If the callback is called on a relaxation and the cut limit is
            not reached
140         if isFractional && cutsCount <= MaxCutsCount
141
142             # Get the solution
143             CPLEX.load_callback_variable_primal(cb_data, context_id)
144
145             # Get the current value of z
146             zBar = callback_value.(cb_data, z)
147
148             # Find a clique
149             newClique = cliqueSeparation(instance, zBar)
150
151             if length(newClique) > 0
152                 #println("(", cutsCount, "/", maxCutsCount, ") Found clique
                    : ", newClique)
153                 cstr = @build_constraint(sum(z[u] for u in newClique) <=
                    instance.p)
154                 MOI.submit(model, MOI.UserCut(cb_data), cstr)
155                 cutsCount += 1
156             else
157                 #println("No clique found")
158             end
159         end
160     end
161
162     # Add the callback to the model
163     MOI.set(model, CPLEX.CallbackFunction(), callbackClique)
164
165 end

```

```

166
167
168     return model, dVariables
169 end

1 function solveQuadratic(resParam::ResolutionParam; windowSize = 200)
2     println(resParam)
3     connexity = resParam.connexity
4     time_limit = resParam.time_limit
5     p = resParam.p
6
7     silent = resParam.verbose == 0
8
9     if !silent###silent==0
10         print("Reading the instance...")
11     end
12     startingReading = time()
13     instance = Instance(resParam.instancePath)
14
15     if !silent
16         println(" done in ", round(Int, time() - startingReading), "s")
17     end
18
19     if p != -1
20         instance.p = p###change p in instance,from definition of resParam.
21     end
22
23     startingTime = time()
24
25     elapsedTime = 0
26     greedyIsSiteOpened = nothing
27     if resParam.warmStart && !resParam.isRelaxation
28
29
30         if resParam.useGridInitialization
31             if !silent
32                 print("Computing optimise solution...")
33             end
34             results = solve(resParam)
35             elapsedTime = results["resolutionTime"]
36             greedyIsSiteOpened = results["isSiteOpened"]
37             greedyIsClientCovered = results["isClientCovered"]
38             greedyObjective = results["objective"]
39             if !silent
40                 println(" Found solution of value ", greedyObjective, " in ",
41                     round(Int, time() - startingTime), "s")
42             end
43         else
44             # utilise la solution d'un algorithme glouton en entr e des
45             # formulations
46             if !silent
47                 print("Computing greedy solution...")
48             end
49             greedyResults = solveGreedy(instance, isConnected = true,
50                 isStochastic = false, p = instance.p, time_limit = resParam.
51                 time_limit)
52
53             elapsedTime = greedyResults["resolutionTime"]
54             greedyIsSiteOpened = greedyResults["isSiteOpened"]
55             greedyIsClientCovered = greedyResults["isClientCovered"]
56             greedyObjective = greedyResults["objective"]
57             #greedyClientCoverage= greedyResults["clientCoverageMatrix"]
58             if !silent
59                 println(" Found solution of value ", greedyObjective, " in ",

```

```

        round(Int, time() - startingTime), "s")
56     end
57   end
58 end
59
60 remainingTime = resParam.time_limit - elapsedTime
61
62 if resParam.time_limit == -1
63   remainingTime = -1
64 end
65
66 results = Dict{String, Any}()
67
68 # If the relaxation is tightened using at least one family of inequalities
69 if length(resParam.cpIneqFamilies) > 0
70
71   if remainingTime != -1
72     remainingTime = time_limit - (time() - startingTime)
73   end
74
75   relaxedModel, relaxedDVariables = getQuadraticModel(instance,
76     isRelaxation = true, threads = resParam.threads, maxCutsCount =
77     resParam.maxCutsCount, connexity=resParam.connexity)
78
79   results["cpTime"], results["cpSeparationTime"], results["rrBeforeCP"],
80     results["rrAfterCP"], results["cpCutsAdded"] = cuttingPlane(instance
81     , relaxedModel, resParam.cpIneqFamilies, relaxedDVariables,
82     maxCutsAdded = resParam.cpMaxCutsCount, time_limit = round(Int,
83     remainingTime / 2))
84
85 end
86
87 if remainingTime != -1
88   remainingTime = time_limit - (time() - startingTime)
89 end
90
91 ## Get the model
92 creationTime = time()
93 if !silent
94   print("Creating the model...")
95 end
96 model, dVariables =getQuadraticModel(instance, time_limit = round(Int,
97   remainingTime), isRelaxation = resParam.isRelaxation, threads = resParam
98   .threads, useCallback = connexity == :QuadCliqueCallback, maxCutsCount =
99   resParam.maxCutsCount, connexity= resParam.connexity)
100
101 if !silent
102   println(" done in ", round(Int, time() - creationTime), "s")
103 end
104
105 if length(resParam.bcIneqFamilies) > 0 && !resParam.isRelaxation
106   addCallback(instance, model, resParam.bcIneqFamilies, dVariables)
107 end
108
109 # Add the inequalities found during the cutting plane
110 if !resParam.isRelaxation
111   for family in resParam.cpIneqFamilies
112     for cut in family.addedCuts
113       add_constraint(model, family.fGetCut(family, model, instance,
114         cut, dVariables))
115     end
116   end
117
118   if !haskey(results, "ciIneqCount")

```

```

108         results["ciIneqCount"] = length(family.addedCuts)
109     else
110         results["ciIneqCount"] += length(family.addedCuts)
111     end
112 end
113 end
114
115 if resParam.ciIneq
116     ciResults = nothing
117     if !haskey(dVariables, :z)
118         ciResults = generateAllCouplesICConstraints(instance, model,
119             dVariables[:z2])
120     else
121         ciResults = generateAllCouplesICConstraints(instance, model,
122             dVariables[:z])
123     end
124     results["ciIneqCount"] = ciResults["ciIneqCount"]
125     results["ciNewVarCount"] = ciResults["ciNewVarCount"]
126     results["ciTime"] = ciResults["ciTime"]
127 end
128
129 if resParam.addMaximalCliques
130     if !silent
131         println("Adding maximal clique constraints...")
132     end
133     startingMCCount = time()
134     mcIneqCount = addMaximalCliquesConstraints(instance, model, dVariables
135         [:z], nothing)
136     results["MCinequalitiesTime"] = time() - startingMCCount
137     results["MCinequalitiesCount"] = mcIneqCount
138 end
139
140 if resParam.tightenRelaxation && !resParam.isRelaxation
141     tightenTime = time()
142
143     if !silent
144         println("Tighten the relaxation...")
145     end
146     cliques, firstRelaxation, lastRelaxation = tightenRelaxationByCliques(
147         instance, connexity = :QuadMTZ, p = instance.p, redundancyRatio=
148         resParam.redundancyRatio, time_limit = resParam.time_limit)
149
150     if !silent
151         println("done in ", round(Int, time() - tightenTime), "s with ",
152             length(cliques), " inequalities")
153     end
154
155     @constraint(model, [clique in cliques], sum(dVariables[:z][u] for u in
156         clique) <= instance.p)
157
158     results["initialBound"] = firstRelaxation
159     results["tightenedBound"] = lastRelaxation
160     results["tighteningTime"] = time() - tightenTime
161     results["inequalitiesCount"] = length(cliques)
162 end
163
164 ## Set the warm start by providing the greedy solution
165 if resParam.warmStart && !resParam.isRelaxation
166
167     greedySiteIdx = findall(greedyIsSiteOpened .> 0.5)
168     greedyPositions = [(instance.sitesPositions[j, 1], instance.
169         sitesPositions[j, 2]) for j in greedySiteIdx]

```

```

163     #
164     maxDiffY = maximum(instance.sitesPositions[:, 2]) - minimum(instance.
165         sitesPositions[:, 2])
166     sortIdx = sortperm(greedyPositions, by = x -> x[1] * maxDiffY + x[2])
167     sortedPositions = greedyPositions[sortIdx]
168     sortedSiteIdx = greedySiteIdx[sortIdx]
169
170     # rerange coverage
171     println("greedySiteIdx: ", greedySiteIdx)
172     println("sortIdx: ", sortIdx)
173     println("sortedSiteIdx: ", sortedSiteIdx)
174     println("sortedPositions: ", sortedPositions)
175     # set up warmstart
176     for hId in 1:length(sortedPositions)
177         h = sortedPositions[hId]
178         for i in 1:instance.n
179             # set the coverage of client i by site hId
180             dist2 = (h[1] - instance.clientPositions[i, 1])^2 + (h[2] -
181                 instance.clientPositions[i, 2])^2 + instance.L^2
182             if dist2 <= instance.rCouv^2 + 1e-6
183                 set_start_value(dVariables[:c][i, hId], 1)
184             else
185                 set_start_value(dVariables[:c][i, hId], 0)
186             end
187             end
188             set_start_value(dVariables[:alpha][hId], h[1])
189             set_start_value(dVariables[:beta][hId], h[2])
190             @constraint(model, dVariables[:alpha][hId] >=h[1]-windowSize)
191             @constraint(model, dVariables[:alpha][hId] <=h[1]+windowSize)
192             @constraint(model, dVariables[:beta][hId] <=h[2]+windowSize)
193             @constraint(model, dVariables[:beta][hId] >=h[2]-windowSize)
194         end
195         if haskey(dVariables, :y)
196             set_start_value.(dVariables[:y], 0)
197             set_start_value.(dVariables[:y][sortedSiteIdx], 1)
198         end
199     end
200     results["instancePath"] = resParam.instancePath
201     results["p"] = instance.p
202     results["n"] = instance.n
203     results["m"] = instance.m
204     results["maxCutsCount"] = resParam.maxCutsCount
205
206     if remainingTime != -1
207         remainingTime = resParam.time_limit - (time() - startingTime)
208     end
209
210     if remainingTime >= 5 || remainingTime == -1
211         if remainingTime != -1
212             if resParam.addEigenvalue
213                 set_optimizer_attribute(model, "MSK_DPAR_OPTIMIZER_MAX_TIME",
214                     remainingTime)
215             else
216                 set_optimizer_attribute(model, "CPX_PARAM_TILIM", remainingTime
217                     )
218             end
219         end
220     end
221
222     if silent
223         set_silent(model)
224     end
225 end

```

```

222     ## Solve it
223     optimize!(model)
224
225
226     if !resParam.isRelaxation
227         results["nodeCount"] = JuMP.node_count(model)
228     end
229
230     if primal_status(model) == MOI.FEASIBLE_POINT
231
232         if resParam.displaySolution
233             displaySolution(dVariables, instance, isRelaxation = resParam.
234                             isRelaxation)
235         end
236
237         results["isFeasible"] = true
238         results["objective"] = JuMP.objective_value(model)
239         results["xHAPS"] = JuMP.value.(dVariables[:alpha])
240         results["yHAPS"] = JuMP.value.(dVariables[:beta])
241
242         if haskey(dVariables, :z)
243             results["isClientCovered"] = JuMP.value.(dVariables[:z])
244         else
245             results["isClientCovered"] = maximum(JuMP.value.(dVariables[:z2
246                                                         ]), dims=1)
247         end
248
249         if termination_status(model) == MOI.OPTIMAL
250             results["isOptimal"] = true
251             results["lowerBound"] = results["objective"]
252         else
253             results["isOptimal"] = false
254             results["lowerBound"] = JuMP.objective_bound(model)
255         end
256     else
257         println("No feasible solution found")
258
259         results["isFeasible"] = false
260     end
261 else
262     results["isOptimal"] = false
263 end
264
265 resolutionTime = time() - startingTime
266
267 ## Get the results
268 results["resolutionTime"] = resolutionTime
269
270 return results
271 end

```

Références

- [1] Zacharie Alès, Sourour Elloumi, and Adèle Pass-Lanneau. Algorithmes de placement optimisé de drones pour la conception de réseaux de communication. Rapport non publié.
- [2] Kiril Danilchenko, Michael Segal, and Zeev Nutov. Covering users by a connected swarm efficiently. In *International Symposium on Algorithms and Experiments for Sensor Systems, Wireless Networks and Distributed Robotics*, pages 32–44. Springer, 2020.
- [3] Yoonsoo Kim and Mehran Mesbahi. On maximizing the second smallest eigenvalue of a state-dependent graph laplacian. In *Proceedings of the 2005, American Control Conference, 2005.*, pages 99–103. IEEE, 2005.